

Universitatea Transilvania din Braşov
Facultatea de Matematică şi Informatică
Specializarea: Informatică Aplicată

Experiment: FP8 E4M3FN
vs INT8 Weight-Only Quantization
ViT-Tiny pe ImageNet-1k validation

Student:

Tudose Alexandru

Grupa 10LF333

An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu

Informatică Aplicată

Cuprins

1	Introducere și Motivație	2
1.1	FP8 E4M3FN — format numeric	2
1.2	Diferența față de Faza 0	2
2	Metodologie	2
2.1	Implementare FP8Linear	2
2.2	Configurații	3
3	Rezultate	3
3.1	Acuratețe și degradare	3
3.2	Eroarea de cuantizare (MSE)	5
3.3	Tradeoff acuratețe–memorie	6
4	Analiză și Interpretare	6
4.1	Verdictul numeric	6
4.2	De ce FP8-pt > INT8-pt?	6
4.3	De ce INT8-pc > FP8-pc?	7
4.4	Latența pe Apple M4	7
5	Concluzii	7

1 Introducere și Motivație

Studiul preliminar (Faza 0) a utilizat cuantizarea la `float8_e4m3fn` prin cast direct, fără stocare reală în FP8 — ponderile erau reconvertite imediat la FP32, deci nu exista reducere de memorie. Experimentul de față implementează corect cuantizarea FP8, replicând structura din Faza 2 (INT8 cu stocare reală și dequantizare la forward pass), dar utilizând formatul `float8_e4m3fn` în loc de INT8.

Motivația vine dintr-o sugestie externă: dacă FP8 E4M3FN are o reprezentare a numerelor în virgulă mobilă (exponent + mantisă), ar putea captura mai bine distribuțiile non-uniforme ale ponderilor față de INT8 (reprezentare uniformă pe întregi).

1.1 FP8 E4M3FN — format numeric

Formatul `float8_e4m3fn` (E4M3) utilizează:

- 1 bit semn
- 4 biți exponent (bias = 7)
- 3 biți mantisă
- Domeniu: ± 448 , cu valori speciale (NaN, dar fără Inf)
- Densitate mai mare de valori reprezentabile în apropierea lui 0

Față de INT8 (256 valori uniform distribuite între -128 și 127), FP8 E4M3FN are o distribuție logaritmică — mai mulți biți pentru valorile mici, mai puțini pentru valorile mari. Ponderile rețelelor neurale tind să aibă distribuții peakate în jurul lui 0, ceea ce teoretic avantajează FP8.

1.2 Diferența față de Faza 0

Tabela 1: Comparatie abordări FP8

	Faza 0 (preliminar)	Acest experiment
Stocare ponderi	FP32 (cast temporar)	<code>float8_e4m3fn</code> (reală)
Reducere memorie	Nu	Da ($\approx 3.3\times$)
Cuantizare	Toate straturile	Selectivă (skip norm, head)
Dequantizare	Imediată la cuantizare	La fiecare forward pass
Dataset	CIFAR-10 (fine-tuned)	ImageNet-1k (pretrained)

2 Metodologie

2.1 Implementare FP8Linear

Clasa `FP8Linear` urmează același pattern ca `QuantizedLinear` din Faza 2, cu adaptarea pentru formatul FP8:

$$s = \frac{\text{FP8_MAX}}{\max(|W|)}, \quad q_{ij} = \text{clamp}(W_{ij} \cdot s, -448, 448).to(\text{float8}) \quad (1)$$

La forward pass:

$$\hat{W}_{ij} = q_{ij}.\text{to}(\text{float32}) \cdot \frac{1}{s} \quad (2)$$

Notă hardware: Pe Apple M4, MPS nu suportă operații native cu `float8_e4m3fn`. Buffer-urile sunt stocate pe CPU, dequantizarea se face pe CPU, iar tensorul rezultat este mutat pe MPS pentru matmul. Pe A100/H100, dequantizarea și matmul-ul pot fi efectuate nativ în hardware.

2.2 Configurații

Tabela 2: Cele 5 configurații evaluate

Metodă	Format stocare	Scalare	Straturi
FP32	float32	—	0
INT8-pt	int8	Per-tensor	48
INT8-pc	int8	Per-channel	48
FP8-pt	float8_e4m3fn	Per-tensor	48
FP8-pc	float8_e4m3fn	Per-channel	48

3 Rezultate

3.1 Acuratețe și degradare

Tabela 3: Rezultate complete — FP8 E4M3FN vs INT8

Metodă	Accuracy	Δ FP32	Loss	Mem (MB)
FP32	75.456%	—	0.9420	21.81
INT8-pt	75.134%	−0.322 pp	0.9563	6.62
INT8-pc	75.424%	−0.032 pp	0.9428	6.70
FP8-pt	75.258%	−0.198 pp	0.9511	6.62
FP8-pc	75.356%	−0.100 pp	0.9495	6.70

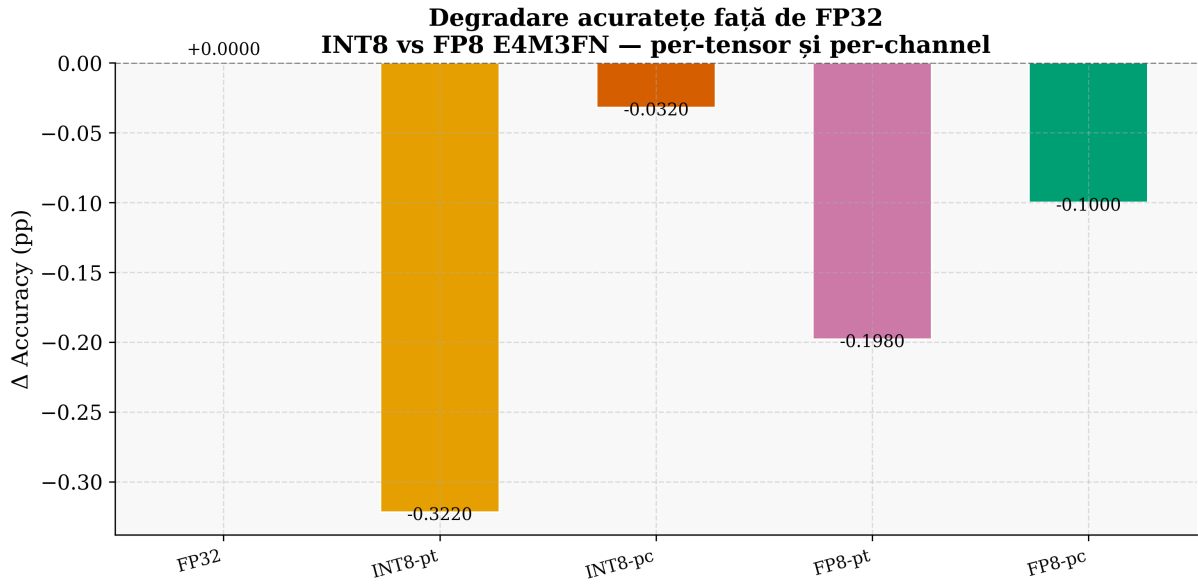


Figura 1: Degradarea de acuratețe față de FP32 pentru toate configurațiile. FP8-pc (-0.100 pp) este mai bun decât INT8-pt (-0.322 pp) dar mai slab decât INT8-pc (-0.032 pp).

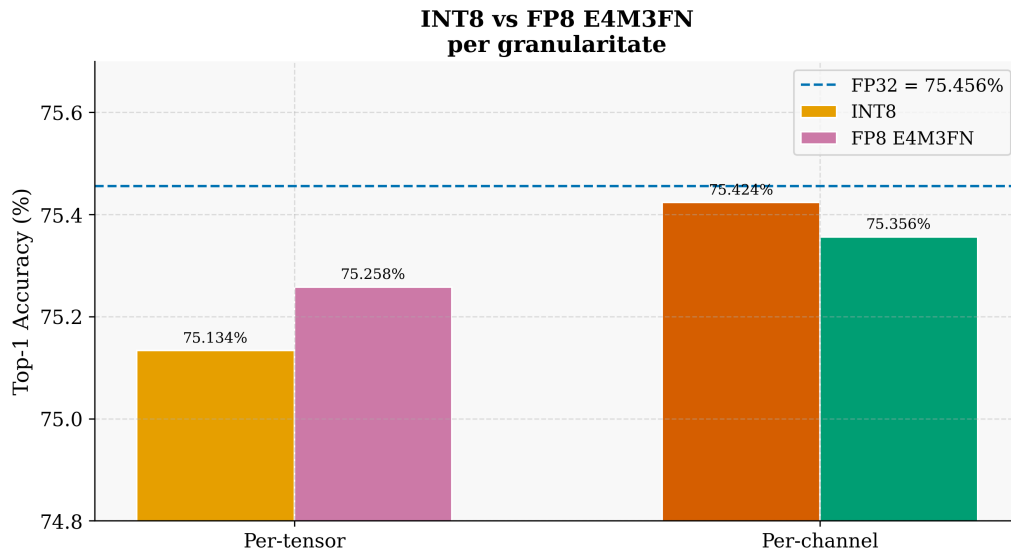


Figura 2: Comparație directă INT8 vs FP8 per granularitate. Per-tensor: FP8 este mai bun cu $+0.124$ pp față de INT8. Per-channel: INT8 este mai bun cu $+0.068$ pp față de FP8.

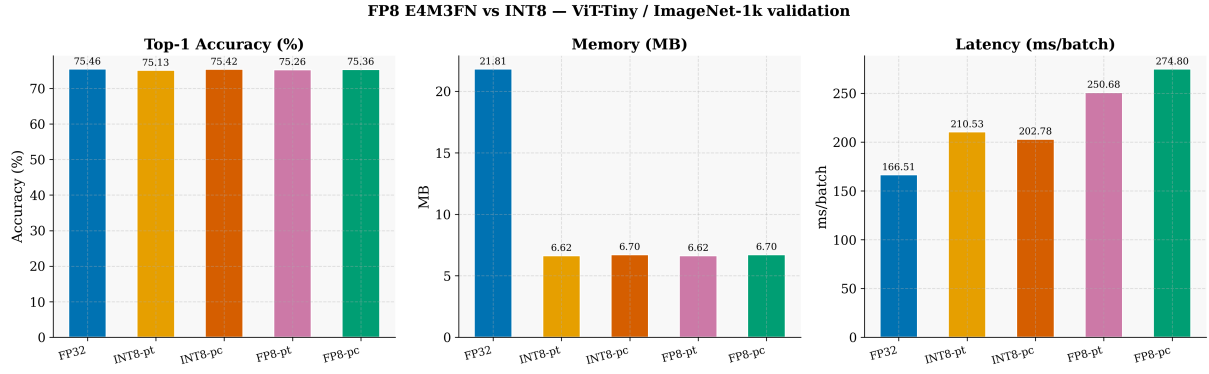


Figura 3: Sumar complet: acuratețe, memorie și latență pentru toate configurațiile. Memoria FP8 și INT8 este identică (aceeași dimensiune de stocare per parametru: 1 byte), latența FP8 este mai mare din cauza dequantizării pe CPU.

3.2 Eroarea de cuantizare (MSE)

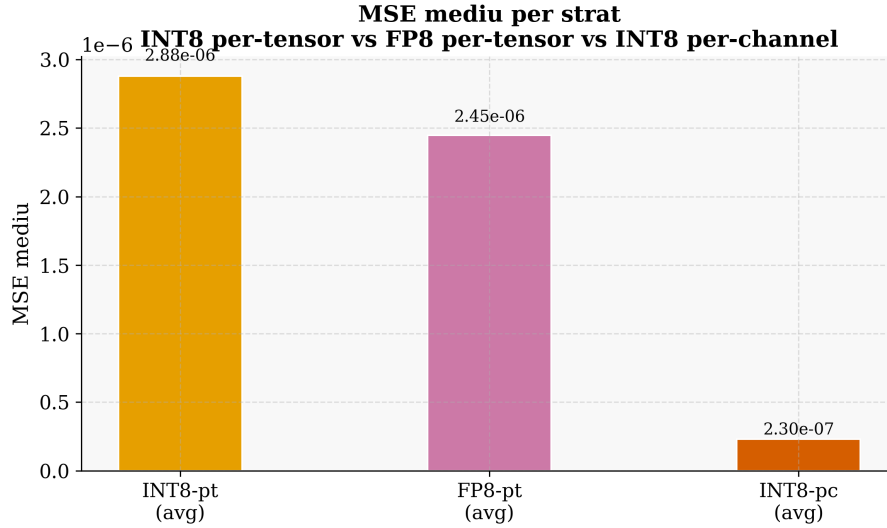


Figura 4: MSE mediu per strat pentru INT8-pt, FP8-pt și INT8-pc. FP8-pt (2.54×10^{-6}) are un MSE similar cu INT8-pt (2.88×10^{-6}), ambele cu un ordin de mărime mai mari decât INT8-pc (2.30×10^{-7}).

Observație cheie privind MSE-ul FP8: Spre deosebire de INT8-pt unde outlierii din block 7 aveau MSE de 3.5×10^{-5} (de $\sim 12\times$ mai mare față de medie), în FP8-pt MSE-ul este uniform: maxim 4.11×10^{-6} (blocks.10.mlp.fc2). FP8 rezolvă natural problema outlierilor prin distribuția logaritmică a valorilor reprezentabile — dar la un cost de acuratețe globală mai mare față de INT8-pc.

3.3 Tradeoff acuratețe–memorie

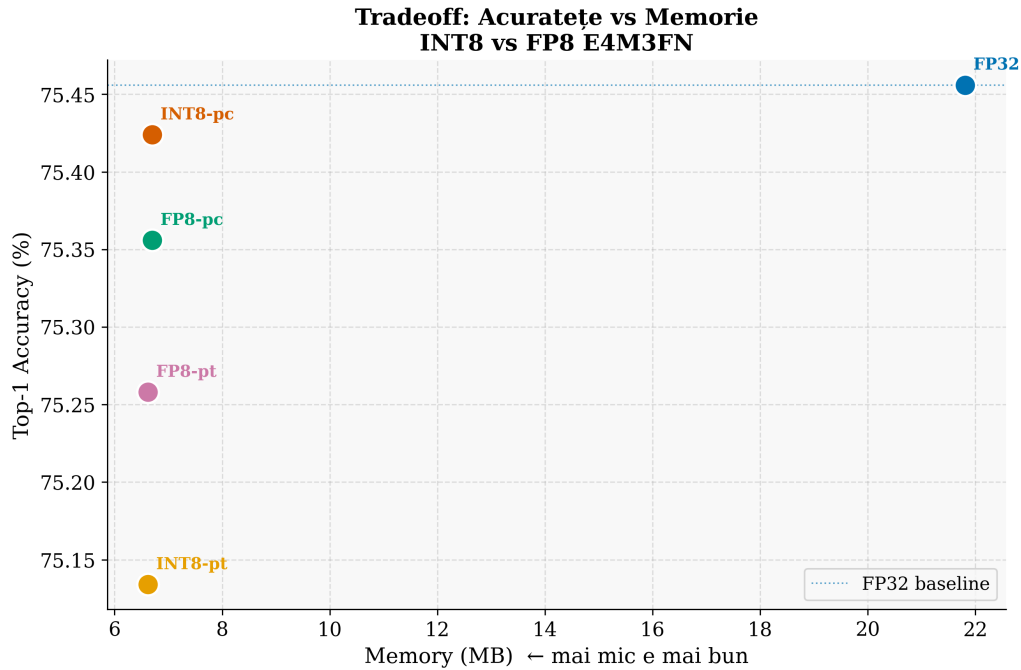


Figura 5: Scatter plot acuratețe vs memorie. INT8-pt și FP8-pt au aceeași memorie (6.62 MB) dar acuratețe diferită. INT8-pc și FP8-pc au aceeași memorie (6.70 MB), INT8-pc fiind superior ca acuratețe.

4 Analiză și Interpretare

4.1 Verdictul numeric

Tabela 4: Verdict: FP8 vs INT8 per granularitate

Comparație	FP8	INT8	Diferență
Per-tensor	75.258%	75.134%	FP8 mai bun cu +0.124 pp
Per-channel	75.356%	75.424%	INT8 mai bun cu +0.068 pp

Rezultatele sunt nuanțate:

- **Per-tensor:** FP8 câștigă (+0.124 pp). Distribuția logaritmică a FP8 compensează parțial problema outlierilor din block 7, pe care INT8 per-tensor o gestionează prost.
- **Per-channel:** INT8 câștigă (+0.068 pp). Per-channel rezolvă deja problema outlierilor prin scale-uri individuale pe canal, iar INT8 oferă mai multă precizie uniformă decât FP8 în această configurație.

4.2 De ce $\text{FP8-pt} > \text{INT8-pt}$?

INT8-pt folosește o singură scalare per matrice. Ponderile din block 7 (`mlp.fc1`, `mlp.fc2`) au outlieri cu valori $\sim 10\times$ mai mari decât restul, ceea ce face ca scala globală să fie prea mare pentru valorile mici — erori de cuantizare ridicate pentru majoritatea parametrilor.

FP8 E4M3FN alocă automat mai mulți biți valorilor mici (distribuție logaritmică). Deși scala e tot globală (per-tensor), formatul FP8 capturează mai bine distribuțiile cu coadă lungă, explicând MSE-ul mai uniform și acuratețea mai bună față de INT8-pt.

4.3 De ce INT8-pc > FP8-pc?

Per-channel elimină problema outlierilor prin scale individuale — INT8-pc a redus MSE-ul cu $12.52\times$ față de INT8-pt (Faza 3). Odată rezolvată problema outlierilor, INT8 devine superior: 256 de valori uniform distribuite per canal oferă mai multă precizie pe distribuțiile gaussiene ale ponderilor decât cei 256 de pași neuniformi ai FP8.

4.4 Latența pe Apple M4

Tabela 5: Latență per configurație (ms/batch)

Metodă	Latență (ms/batch)
FP32	166.5
INT8-pt	210.5
INT8-pc	202.8
FP8-pt	250.7
FP8-pc	274.8

FP8 este semnificativ mai lent pe Apple M4: FP8-pc (274.8 ms) față de INT8-pc (202.8 ms), un overhead de 35%. Motivul: buffer-urile FP8 sunt stocate pe CPU (MPS nu suportă `float8_e4m3fn`), dequantizarea se face pe CPU, iar transferul CPU→MPS adaugă latență la fiecare strat și la fiecare batch. Pe hardware cu suport FP8 nativ (H100 Hopper), situația s-ar inversa — FP8 ar putea fi mai rapid decât INT8.

5 Concluzii

1. **FP8-pt bate INT8-pt cu +0.124 pp:** Distribuția logaritmică a FP8 gestionează mai bine outlierii față de scalarea uniformă INT8 per-tensor, fără a necesita scale per-canal.
2. **INT8-pc rămâne cel mai bun format:** 75.424%, -0.032 pp față de FP32, cu $3.26\times$ reducere de memorie. FP8-pc (75.356%, -0.100 pp) este inferior.
3. **FP8 uniformizează MSE-ul:** Spre deosebire de INT8-pt unde block 7 avea MSE de 3.5×10^{-5} , în FP8-pt MSE-ul maxim este 4.1×10^{-6} — o distribuție mult mai uniformă.
4. **Memoria identică:** FP8 și INT8 ocupă același spațiu (6.62–6.70 MB), deoarece ambele stochează 1 byte per parametru.
5. **FP8 necesită hardware dedicat:** Pe Apple M4, latența FP8 este 35% mai mare față de INT8. Beneficiile reale ale FP8 (speedup hardware + precizie îmbunătățită față de INT8-pt) se manifestă pe GPU-uri Hopper (H100) cu suport nativ pentru FP8 GEMM.

6. **Recomandare generală:** Fără hardware FP8 nativ, INT8-pc rămâne soluția optimă. Cu hardware H100, FP8-pt devine competitiv — mai simplu de implementat decât per-channel și cu acuratețe mai bună decât INT8-pt.

Bibliografie

- [1] P. Micikevicius, D. Stosic, N. Burgess et al., “FP8 Formats for Deep Learning,” 2022. [arXiv:2209.05433](#).
- [2] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. [arXiv:2010.11929](#).
- [3] M. Nagel et al., “A White Paper on Neural Network Quantization,” 2021. [arXiv:2106.08295](#).
- [4] T. Dettmers et al., “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” *NeurIPS*, 2022. [arXiv:2208.07339](#).
- [5] NVIDIA Corporation, “H100 Tensor Core GPU Architecture,” 2022. <https://resources.nvidia.com/en-us-tensor-core>.